# QR Code

## 1. Experiment Objective

Use OpenCV and pyzbar to detect and decode QR codes in the camera image, with support for simultaneous recognition of multiple codes and dynamic parameter tuning.

## 2. Experiment Principle

### 1. Key Terms

| Term | Description |
| --- | --- |
| QR Code | A matrix two-dimensional barcode that can store text/URL/numeric and other information, featuring fast reading and error correction capability |
| pyzbar | A Python barcode/QR code recognition library based on the ZBar backend, providing stable and reliable decoding |
| QRCodeDetector | The QR code detector built into OpenCV (requires the QUIRC library), supporting multi-code/single-code modes |
| Detection Backend | A switchable decoding engine: `pyzbar` (default) / `opencv` / `auto` (prefer pyzbar, fall back to OpenCV) |
| Selection Strategy | The rule for determining the primary code in multi-code scenarios: `largest` (largest area) / `nearest_center` (closest to center) / `first` (the first one) |

### 2. Technical Architecture

```
Terminal 1: start agent + camera + joint models
Terminal 2: start QR code recognition
```

**Data Flow**: Image subscription → image conversion → orientation correction → resize → grayscale conversion → pyzbar / OpenCV detection → draw rectangle boxes + data content labels → real-time display.

## 3. Experiment Steps

### 1. Hardware Preparation

| Hardware | Requirement |
| --- | --- |
| PTZ Version | ☑ Supported |
| No-PTZ Version | ☑ Supported |
| Required Peripherals | ESP32 camera, QR code |

## 2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-10-47-55-372666-yahboom-149737
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [149778]
[INFO] [camera_driver-2]: process started with pid [149780]
[INFO] [robot_state_publisher-3]: process started with pid [149782]
[INFO] [joint_state_publisher-4]: process started with pid [149784]
[INFO] [ekf_node-5]: process started with pid [149803]
[camera_driver-2] [INFO] [1785206876.886150022] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[joint_state_publisher-4] [INFO] [1785206877.007751174] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description topic...
[micro_ros_agent-1] [1785206877.173080] info     | UDPv4AgentLinux.cpp | init                   | running...             | port: 8090
[robot_state_publisher-3] [INFO] [1785206877.754391885] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785206877.754890084] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785206877.754912957] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754918130] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785206877.754924428] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785206877.754929199] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785206877.754935071] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785206877.754939792] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785206877.754944035] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754949951] [robot_state_publisher]: got segment rfwheel_Link
[camera_driver-2] [WARN] [1785206879.871708745] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785206880.046238399] [esp32_ip_camera_publisher]: IP camera stream opened successfully
^[^A
```

## 3. Start QR Code Recognition (Terminal 2)

```
ros2 launch microros_v2_opencv_visual_pkg detect_qr.launch.py
```

## 4. Operation Instructions

**Dual-backend strategy**:

- **pyzbar** (default): grayscale conversion → `pyzbar.decode()`, outputs payload, type, rectangle box, and polygon corners, with high stability
- **OpenCV**: tries `detectAndDecodeMulti` (multi-code) first, falls back to `detectAndDecode` (single-code)
- **auto**: tries pyzbar first, then OpenCV if no result

**Dynamic parameter tuning**:

```
ros2 param list /detect_qr_node
ros2 param set /detect_qr_node qr_select_strategy nearest_center
```

Modify parameters such as detection backend, image orientation, and window size at runtime without restarting.

**Interaction**: Press `q` or `ESC` to exit.

## 5. How to Stop

Press `Ctrl+C` to stop the terminals one by one.

# 4. Experiment Observations

- Place a QR code in the camera's field of view; the code area is marked with a colored rectangle and the decoded content is displayed above it
- In multi-code scenarios, each code is annotated independently, and the primary code (largest area / closest to center) is highlighted

# 5. Program Analysis

> Source code paths:
>
> - Launch: `~/microros_ws/src/microros_v2_opencv_visual_pkg/launch/detect_qr.launch.py`
> - Node: `~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/detect_qr.py`

## 1. Core Node Analysis

`DetectQRNode` implements dual-backend QR code detection: pyzbar first, with OpenCV as the fallback.

```
# Source file:
~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/de
tect_qr.py
class DetectQRNode(Node):
    def __init__(self):
        super().__init__('detect_qr_node')
        self.declare_parameter('qr_detector_backend', 'pyzbar')
        self.declare_parameter('qr_select_strategy', 'largest')
        self.bridge = CvBridge()
```

```
    def detect_with_pyzbar(self, gray):
        from pyzbar.pyzbar import decode as pyzbar_decode
        results = pyzbar_decode(gray)
        codes = []
        for barcode in results:
            payload = barcode.data.decode('utf-8', errors='ignore')
            rect = barcode.rect
            polygon = barcode.polygon
            codes.append({'data': payload, 'rect': rect, 'polygon': polygon})
        return codes

    def image_callback(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
        # Call different detection functions based on the selected backend
        backend = self.get_parameter('qr_detector_backend').value
        if backend == 'pyzbar':
            codes = self.detect_with_pyzbar(gray)
        elif backend == 'opencv':
            codes = self.detect_with_opencv(frame, gray)
        # Draw the detection results
        self.draw_qr_codes(frame, codes)
        cv.imshow('QR Detection', frame)
```

Key logic notes:

- The dual backends are switched at runtime through the `qr_detector_backend` parameter; pyzbar's decoding is more robust than OpenCV's QUIRC
- pyzbar provides `barcode.polygon` with precise positions of multiple corners, while OpenCV only returns a rectangle box
- In multi-code scenarios, the primary code is selected by `qr_select_strategy`: `largest` (largest area, default) / `nearest_center` / `first`

## 2. Key Parameters

> Parameter definition files:
>
> - Launch: `~/microros_ws/src/microros_v2_opencv_visual_pkg/launch/detect_qr.launch.py`
> - Node: `~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/detect_qr.py`

| Parameter | Type | Default Value | Description |
|---|---|---|---|
| `qr_detector_backend` | string | `pyzbar` | Detection backend: `pyzbar` / `opencv` / `auto` |
| `qr_select_strategy` | string | `largest` | Primary code selection strategy: `largest` / `nearest_center` / `first` |
| `topic_name` | string | `/camera/image_raw` | Input image topic |

## 3. Node Description

| Node | Function |
| --- | --- |
| `detect_qr_node` | Dual-backend QR detection and decoding + primary code selection for multiple codes + OpenCV visualization |

# 6. Frequently Asked Questions

| Problem | Cause | Solution |
| --- | --- | --- |
| pyzbar cannot detect | Blurred QR code or insufficient light | Adjust the lighting and make sure the code area is clear |
| OpenCV backend returns no result | The environment lacks the QUIRC library | Switch to the pyzbar backend |